2026-10-22 MySQL基本查询：下

- 1. 表的增删改查与基础查询：下
  - 1.1 与上篇的衔接
    - 1.1.1 本篇范围
    - 1.1.2 测试数据说明
- 2. 更新数据：UPDATE
  - 2.1 UPDATE 基本语法
    - 2.1.1 语句结构
    - 2.1.2 将一列更新为具体值
    - 2.1.3 一次更新多列
  - 2.2 基于原值更新
    - 2.2.1 更新为原值加 30
    - 2.2.2 更新目标在执行语句时确定
  - 2.3 无条件更新全表
    - 2.3.1 将全部语文成绩翻倍
    - 2.3.2 全表更新的安全要求
- 3. 删除数据：DELETE 与 TRUNCATE
  - 3.1 DELETE 基本语法
    - 3.1.1 删除满足条件的记录
    - 3.1.2 删除前确认目标
  - 3.2 删除整表数据
    - 3.2.1 DELETE 会保留表结构与自增计数
  - 3.3 TRUNCATE 截断表
    - 3.3.1 语法与特点
    - 3.3.2 TRUNCATE 示例
    - 3.3.3 DELETE 与 TRUNCATE 对比
    - 3.3.4 关于 MySQL 日志的补充
- 4. 插入查询结果与整表去重
  - 4.1 INSERT ... SELECT
    - 4.1.1 基本语法
  - 4.2 删除重复记录的思路
    - 4.2.1 准备重复数据
    - 4.2.2 创建结构相同的空表
    - 4.2.3 将去重结果插入新表
    - 4.2.4 通过重命名表完成替换
- 5. 聚合函数

- 5.1 聚合函数总览
  - 5.1.1 五个常用函数
- 5.2 COUNT 统计数量
  - 5.2.1 COUNT() 与 COUNT(1)
  - 5.2.2 COUNT(列) 会忽略 NULL
  - 5.2.3 COUNT(DISTINCT ...) 统计不同值的数量
- 5.3 SUM 与 AVG
  - 5.3.1 求和
  - 5.3.2 平均值
- 5.4 MAX 与 MIN
  - 5.4.1 返回英语最高分
  - 5.4.2 返回 70 分以上的数学最低分
  - 5.4.3 聚合值与原始记录

- 6. 分组查询：GROUP BY
  - 6.1 GROUP BY 的作用
    - 6.1.1 基本语法
    - 6.1.2 示例数据说明
  - 6.2 按单个字段分组
    - 6.2.1 每个部门的平均工资和最高工资
  - 6.3 按多个字段分组
    - 6.3.1 按部门和岗位分组
  - 6.4 SELECT 与 GROUP BY 的字段规则
    - 6.4.1 非聚合列必须能够由分组确定
    - 6.4.2 正确理解“分组后可选列”
    - 6.4.3 把分组理解成逻辑子表

- 7. 分组后筛选：HAVING
  - 7.1 HAVING 基本用法
    - 7.1.1 查询平均工资低于 2000 的部门
    - 7.1.2 HAVING 中的别名
  - 7.2 WHERE 与 HAVING 的区别
    - 7.2.1 两者筛选的阶段不同
    - 7.2.2 不要为了使用 HAVING 而使用 HAVING
  - 7.3 带分组统计的执行顺序
    - 7.3.1 主要逻辑顺序
    - 7.3.2 为什么排序时能使用别名，筛选时不能
  - 7.4 MySQL 中的“逻辑表”理解

- 8. 综合练习与速查

- 8.1 讲义实战题目
  - 8.1.1 牛客练习
  - 8.1.2 LeetCode 练习
- 8.2 常用语句速查
  - 8.2.1 更新与删除
  - 8.2.2 插入查询、聚合与分组
- 8.3 高频易错点
  - 8.3.1 更新与删除的易错点
  - 8.3.2 聚合与分组的易错点

1. 表的增删改查与基础查询：下

1.1 与上篇的衔接

1.1.1 本篇范围

上篇已经整理了 `INSERT`、`SELECT`、`WHERE`、`ORDER BY` 和 `LIMIT`。本篇继续整理：

1. `UPDATE`：更新记录。
2. `DELETE` 与 `TRUNCATE`：删除记录或清空整表。
3. `INSERT ... SELECT`：插入查询结果，并借此完成整表去重。
4. `COUNT`、`SUM`、`AVG`、`MAX`、`MIN`：聚合函数。
5. `GROUP BY`：分组统计。
6. `HAVING`：筛选分组后的结果。
7. SQL 执行顺序与综合练习。

1.1.2 测试数据说明

下文的更新、删除和聚合示例承接讲义中的连续操作，因此不同示例中的成绩和记录数量可能已经发生变化。例如删除“孙悟空”后，`exam_result` 不再有该同学的记录；部分更新操作还会改变总分和排序位置。

不能把后一个示例的输出结果直接套回最初刚插入时的表状态。

2. 更新数据：UPDATE

2.1 UPDATE 基本语法

2.1.1 语句结构

```
UPDATE table_name
SET column = expr [, column = expr ...]
[WHERE ...]
[ORDER BY ...]
[LIMIT ...];
```

主要子句的作用：

子句	作用
`UPDATE table_name`	指定要修改的表
`SET ...`	指定要修改的列和新值
`WHERE ...`	选择要修改的记录；省略时表示整表
`ORDER BY ...`	确定更新目标的顺序
`LIMIT ...`	限制本次最多更新多少条记录

更新值既可以是具体常量，也可以是基于原字段的表达式。

2.1.2 将一列更新为具体值

将孙悟空的数学成绩改为 80 分：

```
UPDATE exam_result
SET math = 80
WHERE name = '孙悟空';
```

先查看原数据：

```
SELECT name, math
FROM exam_result
WHERE name = '孙悟空';
```

name	math
孙悟空	78

更新后再查询：

name	math
孙悟空	80

执行结果会显示：

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
```

其中：

- `Rows matched` 表示符合 `WHERE` 条件的记录数。
- `Changed` 表示值确实发生变化的记录数。

2.1.3 一次更新多列

将曹孟德的数学成绩改为 60 分，语文成绩改为 70 分：

```
UPDATE exam_result
SET math = 60,
    chinese = 70
WHERE name = '曹孟德';
```

修改前：

name	math	chinese
曹孟德	84	82

修改后：

name	math	chinese
曹孟德	60	70

多个赋值之间使用逗号分隔。

2.2 基于原值更新

2.2.1 更新为原值加 30

把总分倒数前三的同学的数学成绩各加 30 分。先查看更新前总分最低的三条记录：

```
SELECT
    name,
    math,
    chinese + math + english AS 总分
FROM exam_result
ORDER BY 总分
LIMIT 3;
```

示例结果：

name	math	总分
宋公明	65	170
刘玄德	85	185
曹孟德	60	197

执行更新：

```
UPDATE exam_result
SET math = math + 30
ORDER BY chinese + math + english
LIMIT 3;
```

MySQL 不支持 `math += 30` 这种复合赋值语法，要写成 `math = math + 30`。

2.2.2 更新目标在执行语句时确定

`UPDATE ... ORDER BY ... LIMIT 3` 会根据执行更新前的数据选出 3 条目标记录，再修改它们。

修改后总分可能已经变化，因此不能再直接使用 `ORDER BY 总分 LIMIT 3` 证明“被更新的还是原来倒数三名”。如果要核对结果，应重新查询并计算总分：

```
SELECT
    name,
    math,
    chinese + math + english AS 总分
FROM exam_result
WHERE name IN ('宋公明', '刘玄德', '曹孟德');
```

示例结果：

name	math	总分
曹孟德	90	227
刘玄德	115	215
宋公明	95	200

重新按更新后的总分排序后，倒数三名可能与更新前不同。这是正常现象。

2.3 无条件更新全表

2.3.1 将全部语文成绩翻倍

```
UPDATE exam_result
SET chinese = chinese * 2;
```

因为没有 `WHERE` 子句，所以会对整张表的每一行执行更新。

更新前：

name	chinese
唐三藏	67
孙悟空	87
猪悟能	88
曹孟德	70
刘玄德	55
孙权	70
宋公明	75

更新后：

name	chinese
唐三藏	134
孙悟空	174
猪悟能	176
曹孟德	140
刘玄德	110
孙权	140
宋公明	150

2.3.2 全表更新的安全要求

更新全表前应特别注意：

1. 确认是否真的需要修改所有记录。
2. 先用相同的 `WHERE` 条件执行 `SELECT`，检查目标范围。
3. 必要时先备份数据或在事务中操作。
4. 正式执行前再次检查 `WHERE`、`ORDER BY` 和 `LIMIT`。

3. 删除数据：DELETE 与 TRUNCATE

3.1 DELETE 基本语法

3.1.1 删除满足条件的记录

```
DELETE FROM table_name
[WHERE ...]
[ORDER BY ...]
[LIMIT ...];
```

删除孙悟空的考试成绩：

```
DELETE FROM exam_result
WHERE name = '孙悟空';
```

删除后再次查询：

```
SELECT *
FROM exam_result
WHERE name = '孙悟空';
```

结果为空集。这里的删除是删除记录本身，不是把成绩修改成 0。

3.1.2 删除前确认目标

正式执行 `DELETE` 前，建议先使用相同条件查询：

```
SELECT *
FROM exam_result
WHERE name = '孙悟空';
```

确认待删除记录无误后，再将 `SELECT *` 改为 `DELETE FROM`。

3.2 删除整表数据

3.2.1 DELETE 会保留表结构与自增计数

准备测试表：

```
CREATE TABLE for_delete (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(20)
);
```

插入测试数据：

```
INSERT INTO for_delete (name)
VALUES ('A'), ('B'), ('C');
```

此时记录的 `id` 为 1、2、3。

删除全部数据：

```
DELETE FROM for_delete;
```

表仍然是空的表，结构和索引定义都还在。再次插入数据：

```
INSERT INTO for_delete (name) VALUES ('D');
```

新记录的 `id` 为 4，而不是 1：

id	name
4	D

说明 `DELETE FROM` 虽然删除了全部行，但没有重置 `AUTO_INCREMENT` 计数器。

使用下面的语句可以查看表的定义及当前自增位置：

```
SHOW CREATE TABLE for_delete\G
```

示例输出中会出现：

```
ENGINE=InnoDB AUTO_INCREMENT=5 DEFAULT CHARSET=utf8
```

3.3 TRUNCATE 截断表

3.3.1 语法与特点

```
TRUNCATE [TABLE] table_name;
```

`TRUNCATE` 用于快速清空整张表：

1. 只能清空整表，不能像 `DELETE` 一样按条件删除部分记录。
2. 会保留表结构，但不会保留原数据。
3. 会重置 `AUTO_INCREMENT`。
4. 在 MySQL 中属于 DDL 操作，会隐式提交当前事务，不能按普通 DML 那样回滚。
5. 执行结果的受影响行数通常显示为 0。

3.3.2 TRUNCATE 示例

准备测试表：

```
CREATE TABLE for_truncate (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(20)
);

INSERT INTO for_truncate (name)
VALUES ('A'), ('B'), ('C');
```

清空整表：

```
TRUNCATE for_truncate;
```

再次插入一条数据：

```
INSERT INTO for_truncate (name) VALUES ('D');
```

此时新记录的 `id` 重新从 1 开始：

id	name
1	D

查看表定义时，示例输出会显示：

```
ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8
```

3.3.3 DELETE 与 TRUNCATE 对比

对比项	`DELETE`	`TRUNCATE`
操作对象	可以删除部分或全部行	只能清空整表
条件筛选	支持 `WHERE`	不支持 `WHERE`
表结构	保留	保留
自增计数器	通常保留	重置
事务回滚	事务型表中可随事务回滚	在 MySQL 中按 DDL 处理，不能回滚
受影响行数	返回删除的行数	通常显示 0
使用场景	精确删除指定记录	确定要清空整表时

3.3.4 关于 MySQL 日志的补充

不要仅凭日志名称判断一条语句能否回滚。不同日志承担不同职责：

日志	主要作用
undo log	支持事务回滚和 MVCC
redo log	崩溃恢复与已提交数据持久化
binlog	记录数据变更，用于备份恢复或主从复制

`TRUNCATE` 不能回滚的主要原因是它在 MySQL 中按 DDL 处理并隐式提交，而不是简单因为“没有写日志”。

4. 插入查询结果与整表去重

4.1 INSERT ... SELECT

4.1.1 基本语法

```
INSERT INTO table_name [(column [, column ...])]
SELECT ...;
```

该语法把 `SELECT` 查询出的结果插入目标表，可以用于复制数据、筛选后迁移数据或整理重复记录。

4.2 删除重复记录的思路

4.2.1 准备重复数据

`SELECT DISTINCT` 只能返回一份去重后的查询结果，不会自动修改原表。如果要真正得到一张只含唯一记录的表，可以采用“新建表、去重查询、插入、改名”的流程。

创建原数据表：

```
CREATE TABLE duplicate_table (
    id INT,
    name VARCHAR(20)
);
```

插入重复数据：

```
INSERT INTO duplicate_table VALUES
    (100, 'aaa'),
    (100, 'aaa'),
    (200, 'bbb'),
    (200, 'bbb'),
    (200, 'bbb'),
    (300, 'ccc');
```

表中一共有 6 行，去重后应保留 3 行：

id	name
100	aaa
200	bbb
300	ccc

4.2.2 创建结构相同的空表

```
CREATE TABLE no_duplicate_table LIKE duplicate_table;
```

`CREATE TABLE ... LIKE ...` 会复制表结构，但不会复制原表数据。

4.2.3 将去重结果插入新表

```
INSERT INTO no_duplicate_table
SELECT DISTINCT *
FROM duplicate_table;
```

执行后，`no_duplicate_table` 中有 3 行唯一记录。

4.2.4 通过重命名表完成替换

```
RENAME TABLE duplicate_table TO old_duplicate_table,
             no_duplicate_table TO duplicate_table;
```

这一条 `RENAME TABLE` 语句可以同时完成两张表的改名。相比分别执行两次改名，多表重命名的切换过程更集中，也更适合作为一次统一替换操作。

最终查询：

```
SELECT * FROM duplicate_table;
```

id	name
100	aaa
200	bbb
300	ccc

完整操作可以概括为：

1. 创建一张结构与原表相同的新表。
2. 使用 `SELECT DISTINCT` 得到唯一记录。
3. 使用 `INSERT ... SELECT` 写入新表。
4. 检查新表结果。
5. 使用多表 `RENAME TABLE` 统一替换。

---

5. 聚合函数

5.1 聚合函数总览

5.1.1 五个常用函数

函数	作用
`COUNT([DISTINCT] expr)`	统计数量
`SUM([DISTINCT] expr)`	求和，通常用于数值
`AVG([DISTINCT] expr)`	求平均值，通常用于数值
`MAX([DISTINCT] expr)`	求最大值
`MIN([DISTINCT] expr)`	求最小值

聚合函数会把多行数据归并成一个汇总结果。

5.2 COUNT 统计数量

5.2.1 COUNT(*) 与 COUNT(1)

执行：

```
SELECT 1 FROM exam_result;
```

查询结果会为每一行都生成一个常量 `1`。有多少行，就会出现多少个 `1`。

因此，可以通过统计这些行来获得记录数量：

```
SELECT COUNT(*) FROM students;
SELECT COUNT(1) FROM students;
```

示例结果都是 4，表示 `students` 表中有 4 行。

`COUNT(*)` 和 `COUNT(1)` 都在统计行数，不受列值是否为 `NULL` 影响。

5.2.2 COUNT(列) 会忽略 NULL

统计收集到的 QQ 号数量：

```
SELECT COUNT(qq) FROM students;
```

示例结果为 1，因为 `students` 表中只有一行记录的 `qq` 不为 `NULL`。

对比：

写法	是否统计 NULL
`COUNT(*)`	统计整行数量，包含含 NULL 的行
`COUNT(1)`	统计整行数量，包含含 NULL 的行
`COUNT(qq)`	只统计 `qq IS NOT NULL` 的数量

5.2.3 COUNT(DISTINCT ...) 统计不同值的数量

统计数学成绩记录数：

```sql
SELECT COUNT(math) FROM exam_result;
```

再统计有多少种不同的数学成绩：

```sql
SELECT COUNT(DISTINCT math) AS res
FROM exam_result;
```

按讲义当时的数据状态，示例结果为 6 条成绩、5 种不同分数。该结果承接了前文的更新和删除操作，不要求与初始数据完全一致。

5.3 SUM 与 AVG

5.3.1 求和

统计数学成绩总分：

```sql
SELECT SUM(math) FROM exam_result;
```

讲义示例结果为 569。

如果筛选后没有任何记录，`SUM` 会返回 `NULL`：

```sql
SELECT SUM(math)
FROM exam_result
WHERE math < 60;
```

5.3.2 平均值

统计英语平均分：

```sql
SELECT AVG(english) FROM exam_result;
```

也可以先计算每位同学的总分，再求总分的平均值：

```sql
SELECT AVG(chinese + math + english) AS 平均总分
FROM exam_result;
```

讲义示例结果为 297.5。

5.4 MAX 与 MIN

5.4.1 返回英语最高分

```sql
SELECT MAX(english) FROM exam_result;
```

示例结果为 90。

5.4.2 返回 70 分以上的数学最低分

```sql
SELECT MIN(math)
FROM exam_result
WHERE math > 70;
```

示例结果为 73。

5.4.3 聚合值与原始记录

`MAX(english)` 只返回最高分本身，不自动返回取得最高分的同学姓名。若要同时显示姓名和分数，通常需要先找到最高分，再用它筛选原记录，或使用子查询、窗口函数等后续知识。

6. 分组查询：GROUP BY

6.1 GROUP BY 的作用

6.1.1 基本语法

```sql
SELECT
    column1,
    aggregate_function(expr)
FROM table_name
```

```
WHERE ...
GROUP BY column1;
```

`GROUP BY` 按指定列的不同取值划分逻辑分组，再在每个分组内部执行聚合统计。

例如，各科成绩表可以按学生聚合，员工表可以按部门聚合。

6.1.2 示例数据说明

讲义使用 Oracle 经典的雇员测试数据：

表	含义
`EMP`	员工表
`DEPT`	部门表
`SALGRADE`	工资等级表

以下示例主要使用 `emp` 表中的 `deptno`、`job` 和 `sal` 字段。

6.2 按单个字段分组

6.2.1 每个部门的平均工资和最高工资

```
SELECT
    deptno,
    AVG(sal) AS 平均工资,
    MAX(sal) AS 最高工资
FROM emp
GROUP BY deptno;
```

输出中的每一行代表一个部门：

deptno	平均工资	最高工资
不同部门编号	该部门平均工资	该部门最高工资

如果去掉 `GROUP BY deptno`：

```
SELECT
    MAX(sal) AS 最高,
    AVG(sal) AS 平均
FROM emp;
```

结果会把整个表当成一个组，只返回全公司的最高工资和平均工资。

6.3 按多个字段分组

6.3.1 按部门和岗位分组

统计每个部门、每种岗位的平均工资和最低工资：

```
SELECT
    deptno,
    job,
    AVG(sal) AS 平均工资,
    MIN(sal) AS 最低工资
FROM emp
GROUP BY deptno, job;
```

这里是按 `(deptno, job)` 的组合分组：

- 部门相同、岗位不同，属于不同组。
- 岗位相同、部门不同，也属于不同组。
- 部门与岗位都相同，才属于同一组。

6.4 SELECT 与 GROUP BY 的字段规则

6.4.1 非聚合列必须能够由分组确定

使用 `GROUP BY deptno, job` 时，每个分组内可能包含多位员工，但 `SELECT` 中的 `ename` 是某一位员工的姓名，无法代表整个组，因此会报错：

```sql
SELECT
    ename,
    deptno,
    job,
    MIN(sal) AS 最低,
    AVG(sal) AS 平均
FROM emp
GROUP BY deptno, job;
```

```
mysql> select ename,deptno,job,min(sal) 最低,avg(sal) 平均 from emp group by deptno
,job;
ERROR 1055 (42000): Expression #1 of SELECT list is not in GROUP BY clause and cont
ains nonaggregated column 'scott.emp.ename' which is not functionally dependent on
columns in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by
mysql>
```

典型错误是：

```
ERROR 1055 (42000): Expression #1 of SELECT list is not in
GROUP BY clause and contains nonaggregated column ...
```

原因是 `ename` 既不是聚合函数的结果，也不在 `GROUP BY` 中，每个 `(deptno, job)` 组都可能包含多个姓名，数据库无法确定应该返回哪一个。

6.4.2 正确理解“分组后可选列”

在启用 `ONLY_FULL_GROUP_BY` 时，`SELECT` 中出现的非聚合字段应当出现在 `GROUP BY` 中，或者能够由分组字段唯一确定。

例如下面这样处理，就让 `ename` 也参与分组：

```sql
SELECT
    ename,
    deptno,
    job,
    MIN(sal) AS 最低,
    AVG(sal) AS 平均
FROM emp
GROUP BY deptno, job, ename;
```

但这时分组粒度已经变成“部门 + 岗位 + 姓名”，每个姓名通常只对应一组，不一定还是原来的部门岗位统计。因此，加入字段前要先明确想要的分组粒度。

6.4.3 把分组理解成逻辑子表

分组不是物理上把一张表拆成多张真实表，而是在逻辑上按照分组字段把结果划分为多个子集合：

1. 原表先按 `GROUP BY` 的列值划分组。
2. 每个组内部执行聚合函数。
3. 一个组最终为一个汇总结果。

所以，同一组中能够稳定输出的值，应是分组字段或聚合后的值。不能在每组包含多个不同值时，随意选择其中一个原始字段值。

---

7. 分组后筛选：HAVING

7.1 HAVING 基本用法

7.1.1 查询平均工资低于 2000 的部门

```
SELECT
    deptno,
    AVG(sal) AS deptavg
FROM emp
GROUP BY deptno
HAVING deptavg < 2000;
```

处理顺序是：

1. 按 `deptno` 分组。
2. 在每组内计算平均工资。
3. 使用 `HAVING` 筛选平均值低于 2000 的分组。

7.1.2 HAVING 中的别名

在 MySQL 中，`HAVING` 可以使用 `SELECT` 中定义的别名：

```
SELECT
    deptno,
    AVG(sal) AS myavg
FROM emp
GROUP BY deptno
HAVING myavg < 2000;
```

如果追求 SQL 可移植性，也可以在 `HAVING` 中重复聚合表达式：

```
SELECT
    deptno,
    AVG(sal) AS myavg
FROM emp
GROUP BY deptno
HAVING AVG(sal) < 2000;
```

7.2 WHERE 与 HAVING 的区别

7.2.1 两者筛选的阶段不同

子句	筛选对象	执行阶段	能否筛选聚合结果
`WHERE`	原始记录	分组前	不能直接使用聚合条件
`HAVING`	分组后的结果	分组后	可以

示例：

```
SELECT
    deptno,
    job
FROM emp
WHERE ename <> 'SMITH'
GROUP BY deptno, job;
```

上面先排除 `ename` 为 SMITH 的原始记录，再对剩余记录分组。

而：

```
SELECT
    deptno,
    AVG(sal) AS avg_sal
FROM emp
GROUP BY deptno
HAVING AVG(sal) < 2000;
```

必须先完成分组和聚合，才能判断每个部门的 `AVG(sal)` 是否低于 2000。

7.2.2 不要为了使用 HAVING 而使用 HAVING

如果条件只依赖原始列，应在 `WHERE` 中尽早筛选：

```
SELECT deptno, job
FROM emp
WHERE ename <> 'SMITH'
GROUP BY deptno, job;
```

`WHERE` 先减少参与分组的原始记录，通常比先分组后再用 `HAVING` 过滤更符合查询目的。只有条件依赖聚合结果时，才需要使用 `HAVING`。

7.3 带分组统计的执行顺序

7.3.1 主要逻辑顺序

包含分组和聚合的查询，可以按下面的逻辑顺序理解：

```
FROM / JOIN / ON
    -> WHERE
    -> GROUP BY
    -> HAVING
    -> SELECT
    -> DISTINCT
    -> ORDER BY
    -> LIMIT
```

这一顺序说明：

1. `WHERE` 不能直接使用 `SELECT` 新建的别名。
2. `WHERE` 不能直接使用聚合函数结果。
3. `GROUP BY` 在 `WHERE` 筛选之后执行。
4. `HAVING` 在分组和聚合之后执行。
5. `SELECT` 和 `DISTINCT` 形成最终查询结果。
6. `ORDER BY` 在结果形成后排序。
7. `LIMIT` 最终限制返回行数。

`WITH` 定义的 CTE 应在引用它的查询之前完成定义和求值，不属于上述主查询阶段中“分组之后”的普通步骤。

7.3.2 为什么排序时能使用别名，筛选时不能

结合上篇与本篇可以统一记忆：

子句	能否使用 `SELECT` 别名	原因
`WHERE`	不能	在 `SELECT` 形成别名之前执行
`HAVING`	MySQL 中可以	在分组之后筛选，通常晚于 `SELECT` 表达式求值
`ORDER BY`	可以	排序发生在结果列形成之后

7.4 MySQL 中的“逻辑表”理解

不要只把磁盘上实际存在的表称为表。中间筛选结果、分组后的子集合以及最终查询结果，都可以理解成逻辑上的表。

按顺序理解一条查询：

1. 从数据源表取得数据。
2. 用 `WHERE` 筛选原始记录。
3. 用 `GROUP BY` 划分逻辑分组。
4. 用聚合函数把每组压缩成汇总结果。
5. 用 `HAVING` 筛选汇总后的组。
6. 用 `SELECT` 形成结果列。

7. 用 `ORDER BY` 和 `LIMIT` 完成排序与分页。

掌握单表的增删改查、筛选、分组和聚合后，很多更复杂的 SQL 场景都可以拆解成这些逻辑步骤。

---

8. 综合练习与速查

8.1 讲义实战题目

8.1.1 牛客练习

1. 批量插入数据。
2. 查询所有员工当前薪水，当前条件为 `to_date = '9999-01-01'`；相同薪水只显示一次，并按逆序显示。
3. 查找最晚入职员工的所有信息。
4. 查找入职时间排名倒数第三的员工所有信息。
5. 查找薪水涨幅超过 15 次的员工号 `emp_no` 及涨幅次数 `t`。
6. 查询所有部门当前经理的当前薪水，输出 `dept_no`、`emp_no`、`salary`，当前条件为 `to_date = '9999-01-01'`。
7. 从 `titles` 表按 `title` 分组，找出每组记录数大于等于 2 的标题及数目 `t`。

8.1.2 LeetCode 练习

1. `duplicate-emails`
2. `big-countries`
3. `nth-highest-salary`

这些题目覆盖批量插入、去重、排序、筛选、分组、聚合与 `HAVING`，适合检验 CRUD 和基础查询是否已经连成完整知识链。

8.2 常用语句速查

8.2.1 更新与删除

目标	示例
按条件更新	`UPDATE t SET score = 80 WHERE name = 'A';`
一次更新多列	`UPDATE t SET math = 60, chinese = 70 WHERE id = 4;`
基于原值更新	`UPDATE t SET score = score + 30 WHERE id = 2;`
更新指定前 n 条	`UPDATE t SET score = score + 1 ORDER BY score LIMIT 3;`
按条件删除	`DELETE FROM t WHERE id = 2;`
删除全部记录	`DELETE FROM t;`
清空整表并重置自增	`TRUNCATE TABLE t;`

8.2.2 插入查询、聚合与分组

目标	示例
插入查询结果	`INSERT INTO t2 SELECT * FROM t1;`
统计行数	`SELECT COUNT(*) FROM t;`
统计非空值	`SELECT COUNT(column_name) FROM t;`
统计不同值	`SELECT COUNT(DISTINCT column_name) FROM t;`
求和	`SELECT SUM(score) FROM t;`
求平均	`SELECT AVG(score) FROM t;`
求最值	`SELECT MAX(score), MIN(score) FROM t;`
单列分组	`SELECT deptno, AVG(sal) FROM emp GROUP BY deptno;`
多列分组	`SELECT deptno, job, AVG(sal) FROM emp GROUP BY deptno, job;`
筛选分组结果	`SELECT deptno, AVG(sal) FROM emp GROUP BY deptno HAVING AVG(sal) < 2000;`

8.3 高频易错点

8.3.1 更新与删除的易错点

1. 忘记写 `WHERE`，导致更新或删除整张表。
2. 使用 `+=` 之类不存在的 MySQL 赋值语法。
3. 混淆 `Rows matched` 与 `Changed`。
4. 认为 `DELETE FROM t` 会重置自增编号。
5. 认为 `TRUNCATE` 可以按条件删除部分记录。
6. 认为 MySQL 中的 `TRUNCATE` 可以像普通 DML 一样回滚。

7. 把 `TRUNCATE` 与 `DROP TABLE` 混淆；前者保留表结构，后者删除表本身。

8.3.2 聚合与分组的易错点

1. 以为 `SELECT DISTINCT` 会修改原表。
2. 混淆 `COUNT(*)` 与 `COUNT(column)` 对 NULL 的处理。
3. 以为 `SUM` 在没有匹配记录时会返回 0。
4. 在 `WHERE` 中直接使用聚合函数或 `SELECT` 别名。
5. 在 `GROUP BY` 查询中选择了既未分组、也未聚合的原始字段。
6. 忘记多字段分组是按字段组合划分组。
7. 把本可放在 `WHERE` 的原始记录条件放到 `HAVING`。
8. 认为 `MAX(score)` 会自动返回取得最高分的整条记录。
9. 忘记数据库逻辑执行顺序，导致错误使用别名或过滤条件。